Supplement S1

Example 1: F1-score pathologists vs ML-model

Table of contents

1 Introduction 1

2 Setup 2
- 2.1 Load Packages . . . 2
- 2.2 Functions . . . 2
- 2.3 Load Data . . . 2
- 2.4 Prepare Data . . . 2

3 Bootstrap 3
- 3.1 Average obs. pathologist and ML model (cluster bootstrap) . . . 3
- 3.2 Single pathologist (stratified bootstrap) . . . 4

4 Bayesian estimation 4
- 4.1 Fit Model . . . 4
- 4.2 Get Posterior Draws . . . 5

5 Visualization 7
- 5.1 Figure 1 . . . 7
- 5.2 Supplement Figure S1.1 . . . 8

6 Checks 9
- 6.1 Checks pathologist model . . . 9
  - 6.1.1 Convergence diagnostics . . . 9
  - 6.1.2 Posterior predictive check . . . 10
- 6.2 Checks ML model . . . 12
  - 6.2.1 Convergence diagnostics . . . 12
  - 6.2.2 Posterior predictive check . . . 12

1 Introduction

Example 1 of the paper "Confidence intervals for the F1-score in medical object detection: comparing bootstrap and Bayesian multilevel methods".

Code and data: https://github.com/ndevln/f1-score-ci-object-detection (data: `S1_Data_Pathologists.csv`).

This script estimates the F1-score and its 95% confidence/ credible interval for mitotic figure detection on the Bertram et al. (2021) dataset (https://doi.org/10.1177/03009858211067478): 23 pathologists and one ML model, each evaluated on the same 40 images.

Two methods, each for two estimands:

- Methods: bootstrap and Bayesian multilevel model (binomial + Poisson).
- Estimands: pooled (sum TP/FP/FN over the images, then compute the metric) and grouped (compute the metric per image, then average it).

For the pathologists there are two targets:

- Average of the 23 pathologists: the performance of the observed pathologists, averaged over the images
- Single pathologist: the performance of one new pathologist from the population of pathologists (prediction interval).

All intervals are 95% quantile intervals (QI, percentile interval), the point estimate is the median of the bootstrap or posterior draws.

2 Setup

2.1 Load Packages

```
# Seed (Article number of the Bertram et al. paper)
set.seed(211067478)

# Data manipulation and visualisation
library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(ggdist)

# Bayesian packages
library(brms)       # Fits the multilevel models via Stan
library(tidybayes)  # median_qi()/epred_draws() helpers for posterior summaries
```

2.2 Functions

The F1-score is computed from the counts (paper eq. 3):

$$F_1 = \frac{2TP}{P + TP + FP}, \qquad P = TP + FN$$

Unlike the F1-score from the harmonic mean of precision and recall this formula is defined if TP = 0.

`format_result()` prints a point estimate with its interval as "0.68 (0.67 - 0.70)".

```
calc_F1_score <- function(TP, FP, P) {
  2 * TP / (P + TP + FP)
}

format_result <- function(value, .lower, .upper) {
  fmt <- scales::label_number(accuracy = 0.01, trim = F)
  glue::glue("{fmt(value)} ({fmt(.lower)} - {fmt(.upper)})")
}
```

2.3 Load Data

```
# One row per (model/rater, slide): the per-image confusion-matrix counts
# TP (true positives), FP (false positives), FN (false negatives).
df_pathologists <- readr::read_csv("S1_Data_Pathologists.csv")
```

2.4 Prepare Data

All 40 images contain objects (P >= 1), so no image has to be excluded (paper "Case A").

```
df_pathologists |>
  mutate(
    rater_ID = as.factor(rater_ID),
    GroundTruth = TP + FN,    # Sensitivity/recall trials: all true objects (Positives P)
    # Per-image point metrics
    Precision = pmax(0, (TP/(TP + FP)), na.rm = T), # Set to 0 if TP + FP = 0
    Recall = (TP/(TP + FN)),
    F1 = calc_F1_score(TP, FP, GroundTruth)
  ) -> df_pathologists_prep
```

3 Bootstrap

3.1 Average obs. pathologist and ML model (cluster bootstrap)

The images are resampled with replacement (K = 10,000 resamples), each resampled image with the rows of all 23 pathologists (cluster bootstrap over images). The interval refers to the average of these 23 pathologists. For the ML model (one rater) this is the ordinary image-level bootstrap. Both estimands are computed from the same resampled images:

- Pooled: sum TP, FP and FN over all resampled images (and pathologists) first, then compute the metrics.
- Grouped: average the per-image metrics

```
K = 10000

df_pathologists_prep |>
  distinct(model, slide) |>
  # One bootstrap "study" per .draw
  cross_join(tibble(.draw = 1:K)) |>
  group_by(model, .draw) |>
  # Resample images with replacement up to the number of images
  slice_sample(prop = 1, replace = TRUE) |>
  ungroup() |>
  # Every resampled image with the rows of all pathologists (clustered bootstrap)
  left_join(df_pathologists_prep, by = c("model", "slide"),
            relationship = "many-to-many") |>
  group_by(model, .draw) |>
  summarise(
    # POOLED
    Precision_pooled = pmax(0, sum(TP) / sum(TP + FP), na.rm = T), # 0 if TP + FP = 0
    Recall_pooled = sum(TP) / sum(TP + FN),
    F1_pooled = calc_F1_score(sum(TP), sum(FP), sum(TP + FN)),
    # GROUPED
    Precision_grouped = mean(Precision),
    Recall_grouped = mean(Recall),
    F1_grouped = mean(F1),
    .groups = "drop"
  ) |>
  # Long format: one row per (draw, metric, averaging)
  pivot_longer(matches("_(pooled|grouped)$"), names_to = c("name", "averaging"),
               names_sep = "_") |>
  mutate(
    method = "Bootstrap",
    group = if_else(model == "ML-Model", "ML model", "average obs. pathologist")
  ) -> draws_boot

draws_boot |>
  group_by(method, group, averaging, name) |>
  median_qi(value) |>
  arrange(group, name) |>
  mutate(format = format_result(value, .lower, .upper)) |>
  select(method, group, averaging, name, format)
```

method	group	averaging	name	format
Bootstrap	ML model	grouped	F1	0.80 (0.76 - 0.83)
Bootstrap	ML model	pooled	F1	0.82 (0.79 - 0.85)
Bootstrap	ML model	grouped	Precision	0.83 (0.79 - 0.86)
Bootstrap	ML model	pooled	Precision	0.84 (0.80 - 0.88)
Bootstrap	ML model	grouped	Recall	0.80 (0.75 - 0.85)
Bootstrap	ML model	pooled	Recall	0.81 (0.76 - 0.85)
Bootstrap	average obs. pathologist	grouped	F1	0.63 (0.59 - 0.66)
Bootstrap	average obs. pathologist	pooled	F1	0.68 (0.64 - 0.72)
Bootstrap	average obs. pathologist	grouped	Precision	0.74 (0.70 - 0.78)
Bootstrap	average obs. pathologist	pooled	Precision	0.77 (0.71 - 0.82)
Bootstrap	average obs. pathologist	grouped	Recall	0.62 (0.57 - 0.65)
Bootstrap	average obs. pathologist	pooled	Recall	0.62 (0.57 - 0.65)

3.2 Single pathologist (stratified bootstrap)

The images are resampled within each pathologist (stratified by pathologist), and the metrics are computed per pathologist and draw. The draws of all 23 pathologists are combined into one distribution, so it spans the between-pathologist spread.

```
df_pathologists_prep |>
  filter(model == "Pathologists") |>
  select(rater_ID, TP:FN, Precision:F1) |>
  cross_join(tibble(.draw = 1:K)) |>
  # Stratify by pathologist: resampling happens within each rater_ID
  group_by(rater_ID, .draw) |>
  # Resample images with replacement up to the number of images
  slice_sample(prop = 1, replace = TRUE) |>
  summarise(
    # POOLED
    Precision_pooled = pmax(0, sum(TP) / sum(TP + FP), na.rm = T), # 0 if TP + FP = 0
    Recall_pooled = sum(TP) / sum(TP + FN),
    F1_pooled = calc_F1_score(sum(TP), sum(FP), sum(TP + FN)),
    # GROUPED
    Precision_grouped = mean(Precision),
    Recall_grouped = mean(Recall),
    F1_grouped = mean(F1),
    .groups = "drop"
  ) |>
  # Long format: one row per (pathologist, draw, metric, averaging)
  pivot_longer(matches("_(pooled|grouped)$"), names_to = c("name", "averaging"),
               names_sep = "_") |>
  mutate(method = "Bootstrap", group = "single pathologist") -> draws_boot_single

# The draws of all pathologists combined into one distribution
draws_boot_single |>
  group_by(method, group, averaging, name) |>
  median_qi(value) |>
  arrange(name) |>
  mutate(format = format_result(value, .lower, .upper)) |>
  select(method, group, averaging, name, format)
```

method	group	averaging	name	format
Bootstrap	single pathologist	grouped	F1	0.64 (0.45 - 0.77)
Bootstrap	single pathologist	pooled	F1	0.68 (0.52 - 0.79)
Bootstrap	single pathologist	grouped	Precision	0.76 (0.48 - 0.92)
Bootstrap	single pathologist	pooled	Precision	0.83 (0.54 - 0.95)
Bootstrap	single pathologist	grouped	Recall	0.65 (0.33 - 0.81)
Bootstrap	single pathologist	pooled	Recall	0.62 (0.36 - 0.82)

4 Bayesian estimation

4.1 Fit Model

Binomial + Poisson model with two joint sub-models:

- Sensitivity/Recall: TP successes out of GroundTruth (= TP + FN) trials (binomial).
- False positives: FP count per image (Poisson, log link).

Precision follows from both parts: recall × P / (recall × P + FP).

Random effects:

- `(1|slide)`: per-image difficulty.
- `(1|r|rater_ID)`: per-pathologist effect; the shared label `r` correlates it across the two sub-models and represents each pathologist's cutoff.

```r
bf_tp <- bf(TP | trials(GroundTruth) ~ 1 + (1|slide) + (1|r|rater_ID), family = binomial())
bf_fp <- bf(FP ~ 1 + (1|slide) + (1|r|rater_ID), family = poisson())

# 4 chains, 2000 warmup of 10,000 iterations, default brms weakly-informative priors.
brms_pathologists_model <- brm(bf_tp + bf_fp,
                               data = df_pathologists_prep |> filter(model == "Pathologists"),
                               chains = 4, warmup = 2000, iter = 10000, cores = 4,
                               # Rater SD hard to estimate when the number of raters is low.
                               # Increase adapt_delta and max_treedepth to improve convergence
                               control = list(adapt_delta = 0.95, max_treedepth = 12),
                               seed = 211067478,
                               refresh = 0) # No sampling progress in the output for PDF

# ML model has a single rater, so there is no rater_ID effect
bf_tp <- bf(TP | trials(GroundTruth) ~ 1 + (1|slide), family = binomial())
bf_fp <- bf(FP ~ 1 + (1|slide), family = poisson())

brms_ml_model <- brm(bf_tp + bf_fp,
                     data = df_pathologists_prep |> filter(model == "ML-Model"),
                     chains = 4, warmup = 2000, iter = 10000, cores = 4,
                     seed = 211067478,
                     refresh = 0)
```

4.2 Get Posterior Draws

The marginal estimates refer to the population of images that the observed images represent (CAVE: if images are a convenience sample). For every posterior draw:

1. Expected TP and FP of every observed image with its fitted image effect (trials = 1 gives the recall, expected TP = recall × P):
   - average of the observed pathologists (`get_epred_draws_observed()`): with the fitted effects of the 23 pathologists; for the ML model without rater level,
   - single pathologist (`get_epred_draws_with_path_re()`): one new pathologist whose effects are drawn from the fitted distribution of the pathologist effects (`sample_new_levels = "gaussian"`, prediction interval).
2. Bayesian bootstrap weights over the images, Dirichlet(1, …, 1) via `rexp()`, shared by all pathologists of an image.
3. Pooled and grouped metrics with these weights (`transform_epred_calc_F1()`).

```r
# Average of the observed pathologists (or the ML model): their fitted effects on
# every observed image
get_epred_draws_observed <- function(brms_model, ci) {
  epred_draws(brms_model,
              newdata = brms_model$data |> mutate(P = GroundTruth, GroundTruth = 1)) |>
    mutate(ci_type = ci)
}

# Single pathologist: one NEW pathologist on every observed image
get_epred_draws_with_path_re <- function(brms_model) {
  epred_draws(brms_model,
              newdata = distinct(brms_model$data, slide, P = GroundTruth) |>
                mutate(GroundTruth = 1, rater_ID = "new_pathologist"),
              allow_new_levels = TRUE, sample_new_levels = "gaussian") |>
    mutate(ci_type = "single pathologist")
}

# The two sub-models arrive stacked in the .category column (TP / FP). Expected
# counts per cell, Bayesian bootstrap weights over the images and the marginal
# pooled and grouped metrics per posterior draw (long format).
transform_epred_calc_F1 <- function(df) {
  df |>
    ungroup() |>
    select(ci_type, .draw, any_of("rater_ID"), slide, P, .category, .epred) |>
    pivot_wider(names_from = .category, values_from = .epred) |>
    mutate(
      Recall = TP,               # recall of the cell (image x pathologist)
      TP = Recall * P,           # expected TP of the cell
```

```
      Precision = TP / (TP + FP) # expected TP / (expected TP + expected FP)
    ) |>
    # Bayesian bootstrap weights per image, shared by all pathologists of the image
    group_by(ci_type, .draw, slide) |>
    mutate(weight = rexp(1)) |>
    group_by(ci_type, .draw) |>
    summarise(
      # POOLED
      Precision_pooled = sum(weight * TP) / sum(weight * (TP + FP)),
      Recall_pooled = sum(weight * TP) / sum(weight * P),
      F1_pooled = calc_F1_score(sum(weight * TP), sum(weight * FP), sum(weight * P)),
      # GROUPED
      Precision_grouped = weighted.mean(Precision, weight),
      Recall_grouped = weighted.mean(Recall, weight),
      F1_grouped = weighted.mean(calc_F1_score(TP, FP, P), weight),
      .groups = "drop"
    ) |>
    pivot_longer(matches("_(pooled|grouped)$"), names_to = c("name", "averaging"),
                 names_sep = "_")
}
```

```
# Pathologists: average of the observed pathologists and single pathologist
bind_rows(
  get_epred_draws_observed(brms_pathologists_model, ci = "average obs. pathologist"),
  get_epred_draws_with_path_re(brms_pathologists_model)) |>
  transform_epred_calc_F1() -> res_model_path

# ML model: only the observed (single) rater
get_epred_draws_observed(brms_ml_model, ci = "ML model") |>
  transform_epred_calc_F1() -> res_model_ml

bind_rows(res_model_path, res_model_ml) |>
  mutate(method = "Bayesian", group = ci_type) -> draws_bayesian
```

Point estimate (posterior median) and 95% QI per group, estimand and metric:

```
draws_bayesian |>
  group_by(method, group, averaging, name) |>
  median_qi(value) |>
  arrange(group, name) |>
  mutate(format = format_result(value, .lower, .upper)) |>
  select(method, group, averaging, name, format)
```

method	group	averaging	name	format
Bayesian	ML model	grouped	F1	0.78 (0.73 - 0.81)
Bayesian	ML model	pooled	F1	0.82 (0.79 - 0.85)
Bayesian	ML model	grouped	Precision	0.80 (0.75 - 0.85)
Bayesian	ML model	pooled	Precision	0.84 (0.80 - 0.88)
Bayesian	ML model	grouped	Recall	0.79 (0.73 - 0.83)
Bayesian	ML model	pooled	Recall	0.81 (0.76 - 0.85)
Bayesian	average obs. pathologist	grouped	F1	0.63 (0.58 - 0.66)
Bayesian	average obs. pathologist	pooled	F1	0.68 (0.65 - 0.72)
Bayesian	average obs. pathologist	grouped	Precision	0.71 (0.66 - 0.75)
Bayesian	average obs. pathologist	pooled	Precision	0.77 (0.71 - 0.82)
Bayesian	average obs. pathologist	grouped	Recall	0.61 (0.57 - 0.65)
Bayesian	average obs. pathologist	pooled	Recall	0.62 (0.57 - 0.65)
Bayesian	single pathologist	grouped	F1	0.63 (0.43 - 0.74)
Bayesian	single pathologist	pooled	F1	0.69 (0.47 - 0.79)
Bayesian	single pathologist	grouped	Precision	0.73 (0.39 - 0.91)
Bayesian	single pathologist	pooled	Precision	0.83 (0.47 - 0.96)
Bayesian	single pathologist	grouped	Recall	0.62 (0.34 - 0.84)
Bayesian	single pathologist	pooled	Recall	0.62 (0.33 - 0.84)

Observed performance of each of the 23 pathologists (pooled and grouped over the 40 images), shown as points in Figure 1.

```r
df_pathologists_prep |>
  filter(model == "Pathologists") |>
  group_by(rater_ID) |>
  summarise(
    # POOLED
    Precision_pooled = sum(TP) / sum(TP + FP),
    Recall_pooled = sum(TP) / sum(TP + FN),
    F1_pooled = calc_F1_score(sum(TP), sum(FP), sum(TP + FN)),
    # GROUPED
    Precision_grouped = mean(Precision),
    Recall_grouped = mean(Recall),
    F1_grouped = mean(F1)
  ) |>
  pivot_longer(matches("_(pooled|grouped)$"), names_to = c("name", "averaging"),
               names_sep = "_") |>
  cross_join(tibble(group = c("average obs. pathologist", "single pathologist"))) -> df_path_macro
```

5 Visualization

5.1 Figure 1

```r
draws_bayesian |>
  filter(averaging == "pooled") |>
  ggplot(aes(x = group, y = value, fill = group)) +
  # Posterior density + median and 95% QI for each group.
  stat_halfeye(aes(color = group), point_interval = "median_qi", .width = 0.95,
               slab_alpha = 0.5, normalize = "xy", scale = 0.7, show.legend = F) +
  # Observed per-pathologist values of the same estimand as a one-sided beeswarm
  # (the data function picks the averaging of the plotted draws).
  ggbeeswarm::geom_beeswarm(data = \(df) filter(df_path_macro, averaging %in% df$averaging),
                            size = 0.6, cex = 2.5, side = -1L, method = "compactswarm") +
  facet_wrap(~name, ncol = 1, strip.position = "right") +  # one panel per metric
  scale_y_continuous(breaks = (0:10) / 10, expand = c(0, 0)) +
  scale_x_discrete(limits = c("ML model", "single pathologist",
                              "average obs. pathologist")) +
  xlab("") +
  coord_flip(ylim = c(0.2, 1)) +  # horizontal layout, zoom to the informative range
  ggokabeito::scale_color_okabe_ito(name = "pathologists") +  # colour-blind-safe palette
  ggokabeito::scale_fill_okabe_ito(name = "pathologists") +
  theme_bw() +
  theme(legend.position = "top") -> fig_pathologists

fig_pathologists
```

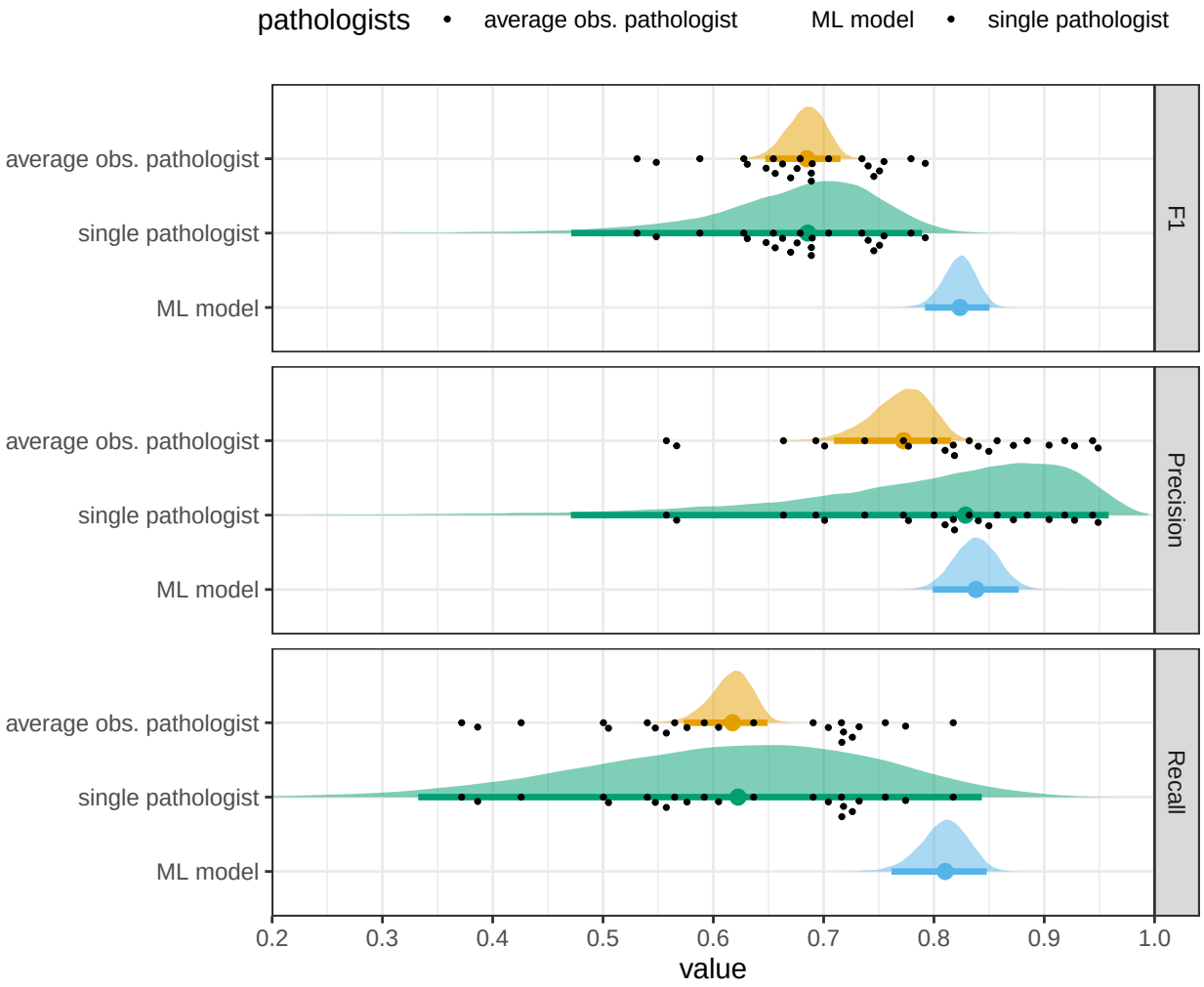
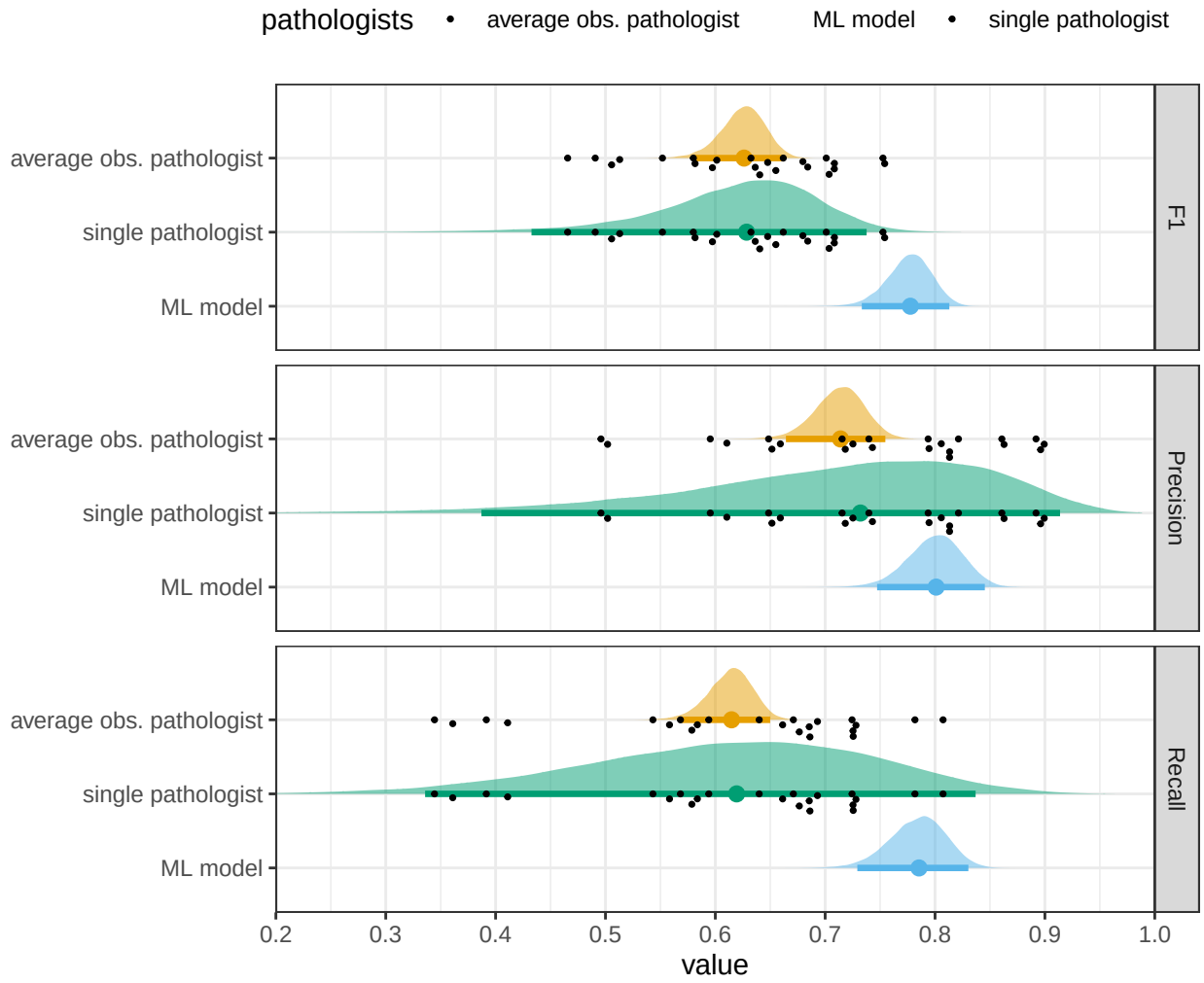


Figure 1: Posterior distributions (pooled estimand) of the ML model, a single new pathologist and the average of the 23 observed pathologists, with the observed pathologists as points.

```
ggsave(fig_pathologists, filename = "Figure_1.pdf", width = 7, height = 5)
```

5.2 Supplement Figure S1.1

```
# Figure 1 with its data replaced by the grouped draws and point estimates
fig_pathologists + filter(draws_bayesian, averaging == "grouped")
```



Supplement Figure S1.1: Posterior distributions (grouped estimand) of the ML model, a single new pathologist and the average of the 23 observed pathologists, with the observed pathologists as points.

6 Checks

6.1 Checks pathologist model

6.1.1 Convergence diagnostics

```
# Convergence over all parameters: max Rhat (should be < 1.01), min bulk and tail ESS
# (should be > 400), number of divergent transitions (should be 0)
df_summary <- posterior::summarise_draws(as_draws_array(brms_pathologists_model), "rhat", "ess_bulk", "ess_tail")
tibble(
  max_rhat     = max(df_summary$rhat, na.rm = TRUE),
  min_ess_bulk = min(df_summary$ess_bulk, na.rm = TRUE),
  min_ess_tail = min(df_summary$ess_tail, na.rm = TRUE),
  divergences  = sum(subset(nuts_params(brms_pathologists_model), Parameter == "divergent__")$Value)
)
```

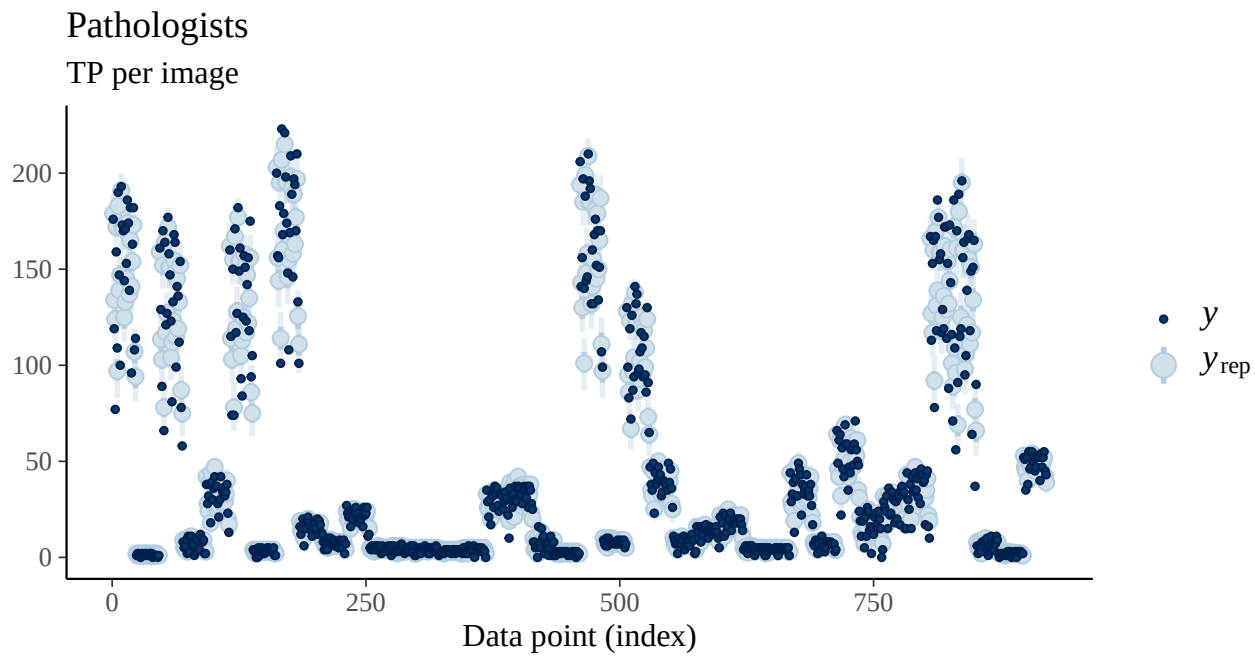
max_rhat	min_ess_bulk	min_ess_tail	divergences
1.001662	3346.383	6020.407	0

```
# Per population-level parameter (intercepts, SDs of the image and pathologist effects,
# correlation of the pathologist effects)
posterior::summarise_draws(as_draws_array(brms_pathologists_model, variable = "^(b|sd|cor)_", regex = TRUE),
                           "rhat", "ess_bulk", "ess_tail")
```

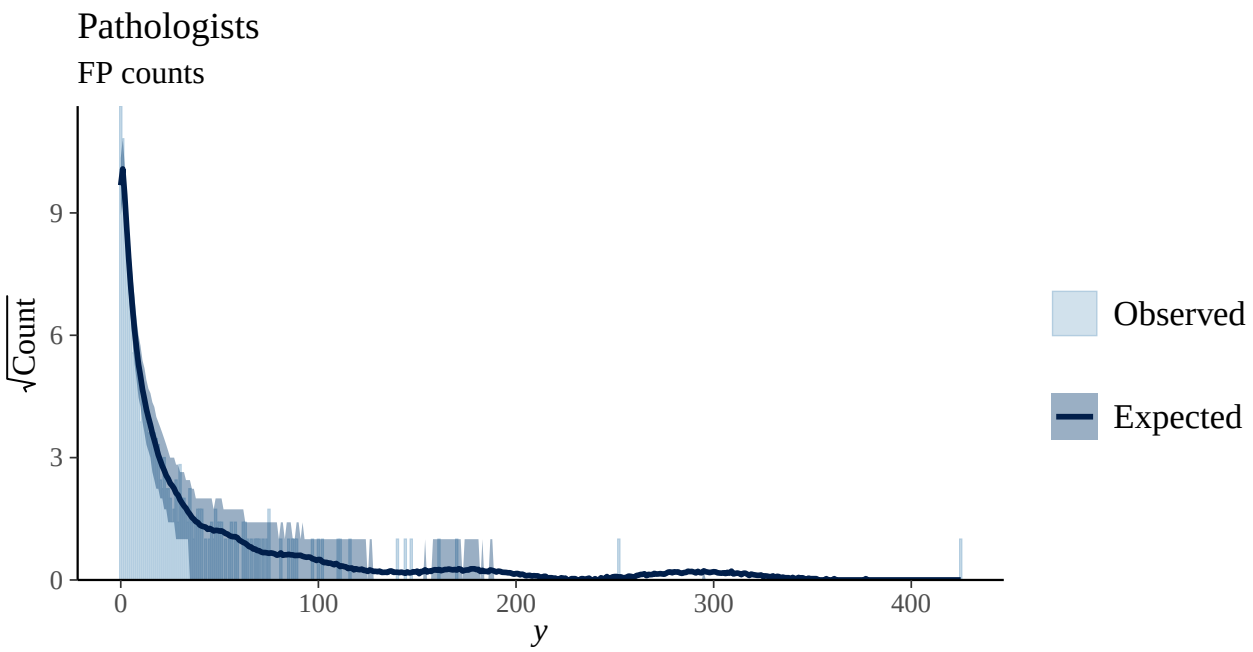
variable	rhat	ess_bulk	ess_tail
b_TP_Intercept	1.001662	3852.944	6400.035
b_FP_Intercept	1.001422	3893.918	6050.849
sd_rater_ID__TP_Intercept	1.000255	4913.830	8963.920
sd_rater_ID__FP_Intercept	1.000578	5032.595	9115.297
sd_slide__TP_Intercept	1.000777	5846.736	10621.946
sd_slide__FP_Intercept	1.000432	3821.341	6256.553
cor_rater_ID__TP_Intercept__FP_Intercept	1.000274	5542.648	9210.127

6.1.2 Posterior predictive check

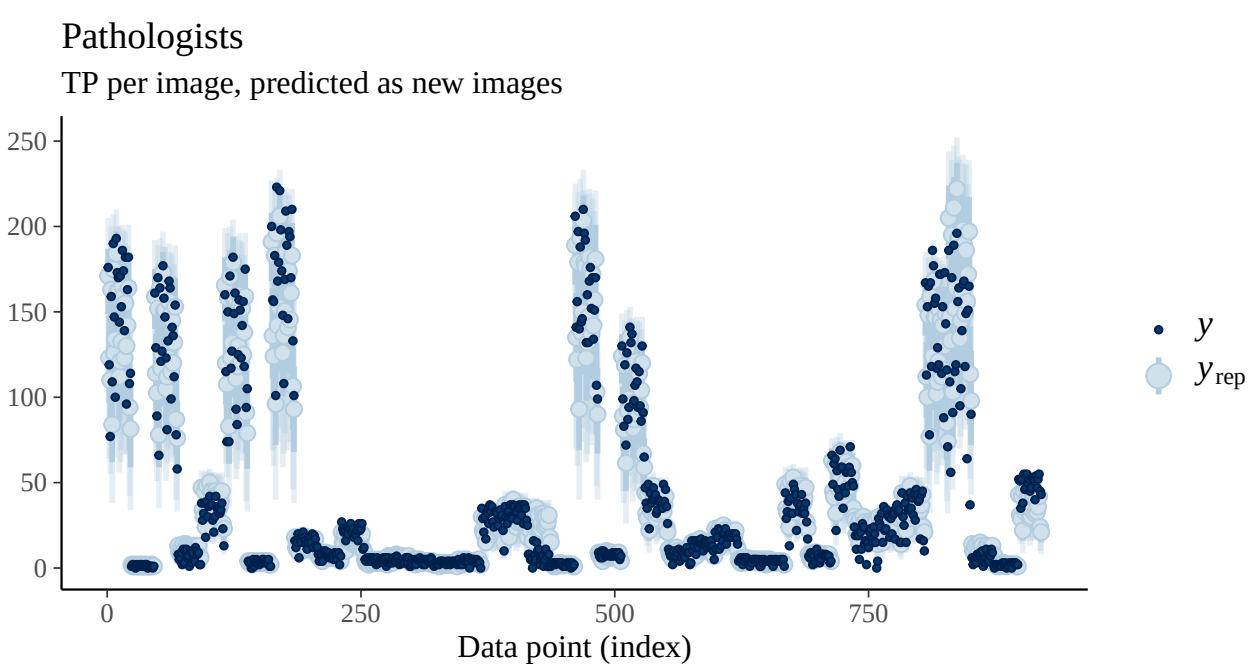
```
# TP: observed TP per image and pathologist (dot) with the 50% / 90% predictive interval
pp_check(brms_pathologists_model, resp = "TP", type = "intervals", ndraws = 1000) +
  ggtitle("Pathologists", "TP per image")
```



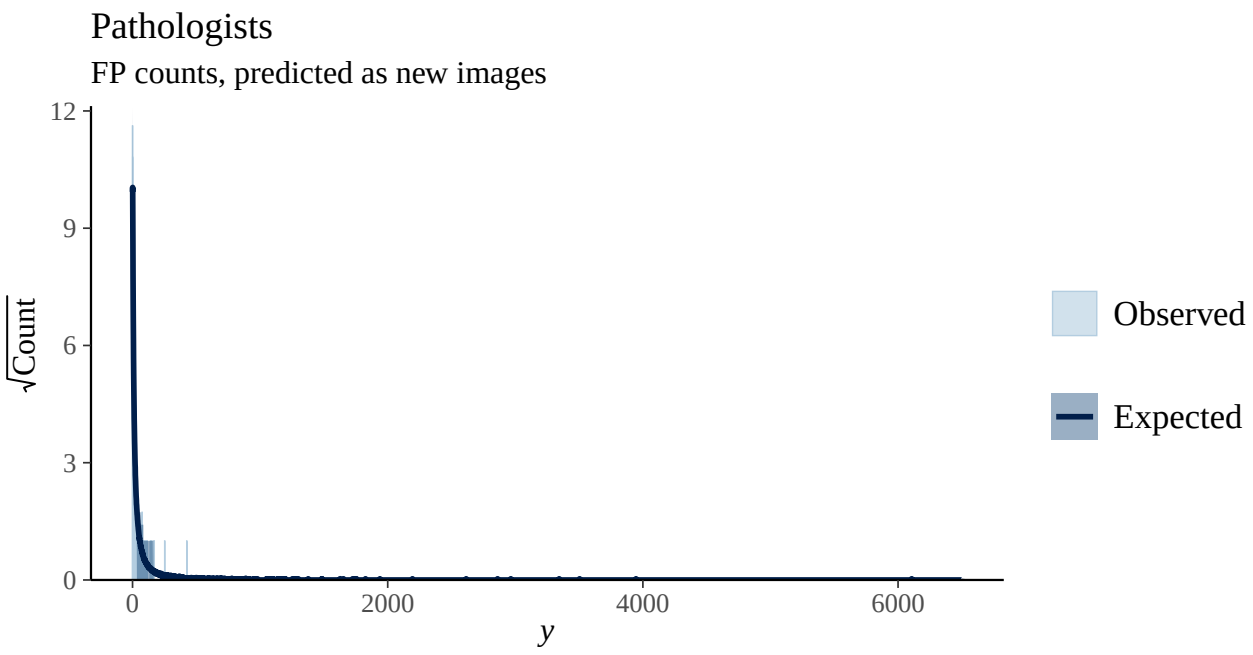

```
# FP: rootogram of the FP counts (observed bars vs. expected frequencies)
pp_check(brms_pathologists_model, resp = "FP", type = "rootogram", ndraws = 1000) +
  ggtitle("Pathologists", "FP counts")
```




```
# TP per image, predicted as new images
pp_check(brms_pathologists_model, resp = "TP", type = "intervals", ndraws = 1000,
         newdata = brms_pathologists_model$data |> mutate(slide = paste0("new_", slide)),
         allow_new_levels = TRUE, sample_new_levels = "gaussian") +
  ggtitle("Pathologists", "TP per image, predicted as new images")
```




```
# FP counts, predicted as new images
pp_check(brms_pathologists_model, resp = "FP", type = "rootogram", ndraws = 1000,
         newdata = brms_pathologists_model$data |> mutate(slide = paste0("new_", slide)),
         allow_new_levels = TRUE, sample_new_levels = "gaussian") +
  ggtitle("Pathologists", "FP counts, predicted as new images")
```



6.2 Checks ML model

6.2.1 Convergence diagnostics

```
# Convergence over all parameters, same diagnostics as for the pathologist model
df_summary <- posterior::summarise_draws(as_draws_array(brms_ml_model), "rhat", "ess_bulk", "ess_tail")
tibble(
  max_rhat     = max(df_summary$rhat, na.rm = TRUE),
  min_ess_bulk = min(df_summary$ess_bulk, na.rm = TRUE),
  min_ess_tail = min(df_summary$ess_tail, na.rm = TRUE),
  divergences  = sum(subset(nuts_params(brms_ml_model), Parameter == "divergent__")$Value)
)
```

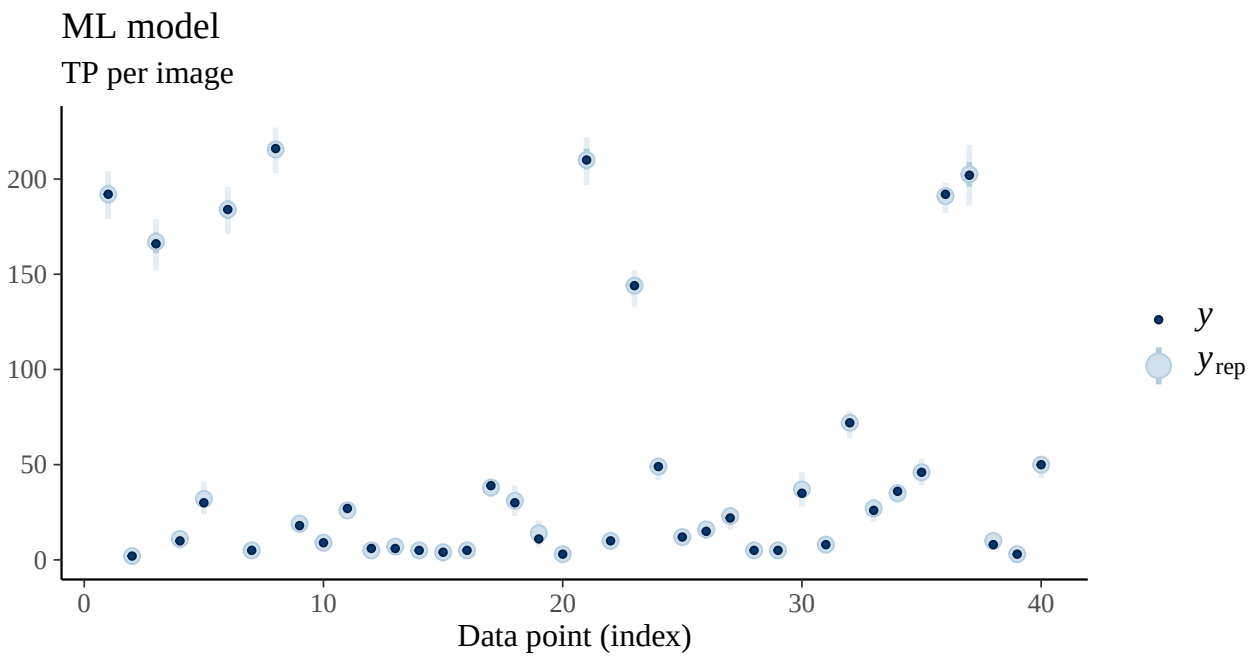
max_rhat	min_ess_bulk	min_ess_tail	divergences
1.0009	4282.192	8476.231	0

```
# Per population-level parameter (intercepts and SD of the per-image effects)
posterior::summarise_draws(as_draws_array(brms_ml_model, variable = "^(b|sd)_", regex = TRUE),
                           "rhat", "ess_bulk", "ess_tail")
```

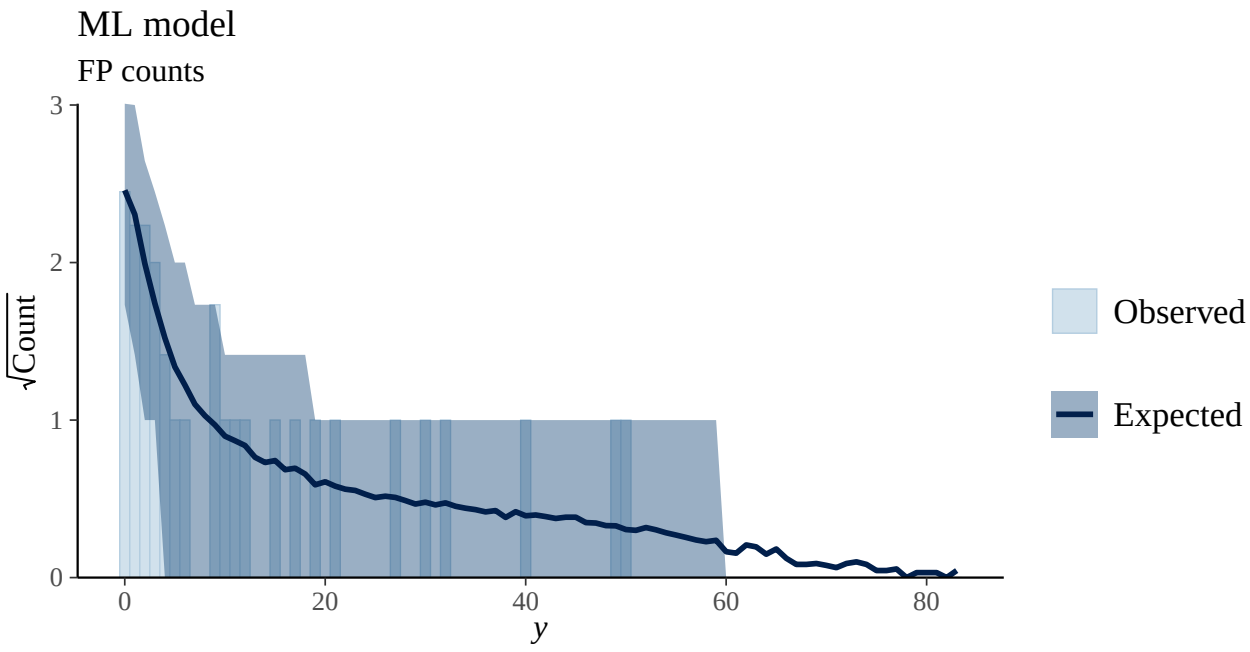
variable	rhat	ess_bulk	ess_tail
b_TP_Intercept	1.000709	6168.279	11482.386
b_FP_Intercept	1.000900	4282.192	8476.231
sd_slide__TP_Intercept	1.000289	8576.678	16206.599
sd_slide__FP_Intercept	1.000549	5936.644	12243.645

6.2.2 Posterior predictive check

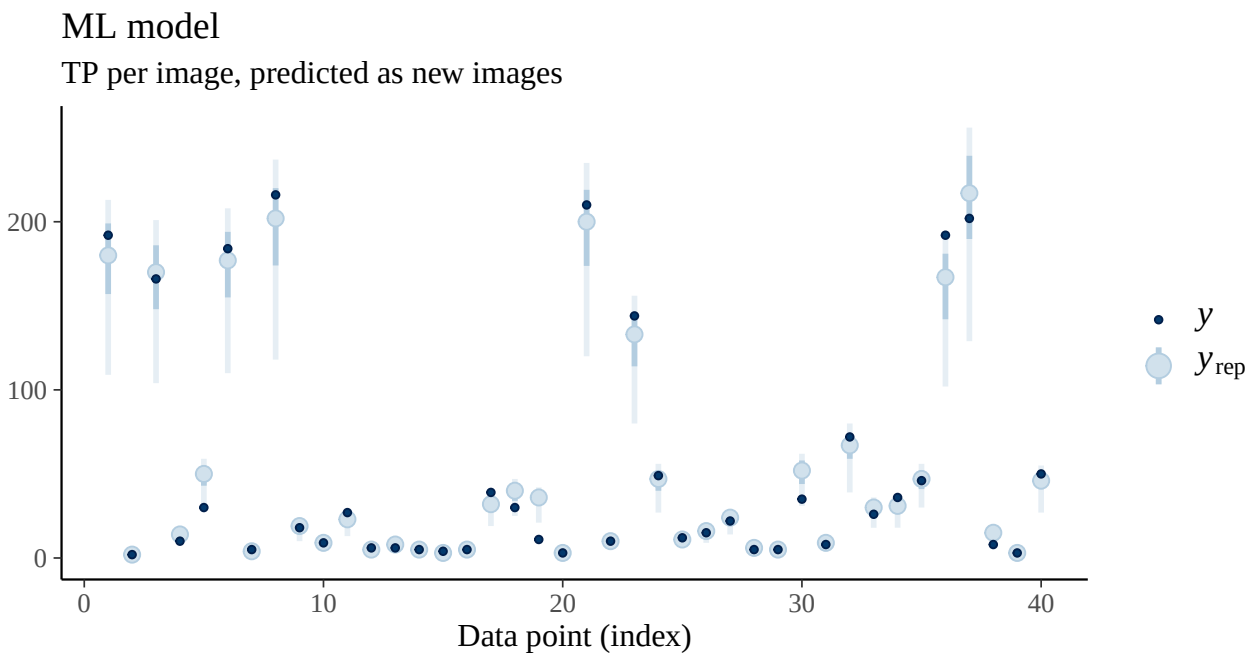
```
# TP: observed TP per image (dot) with the 50% / 90% predictive interval
pp_check(brms_ml_model, resp = "TP", type = "intervals", ndraws = 1000) +
  ggtitle("ML model", "TP per image")
```




```
# FP: rootogram of the FP counts (observed bars vs. expected frequencies)
pp_check(brms_ml_model, resp = "FP", type = "rootogram", ndraws = 1000) +
  ggtitle("ML model", "FP counts")
```




```
# TP per image, predicted as new images
pp_check(brms_ml_model, resp = "TP", type = "intervals", ndraws = 1000,
         newdata = brms_ml_model$data |> mutate(slide = paste0("new_", slide)),
         allow_new_levels = TRUE, sample_new_levels = "gaussian") +
  ggtitle("ML model", "TP per image, predicted as new images")
```




```
# FP counts, predicted as new images
pp_check(brms_ml_model, resp = "FP", type = "rootogram", ndraws = 1000,
         newdata = brms_ml_model$data |> mutate(slide = paste0("new_", slide)),
         allow_new_levels = TRUE, sample_new_levels = "gaussian") +
  # zoom to twice the largest observed FP count
  coord_cartesian(xlim = c(0, 2 * max(brms_ml_model$data$FP))) +
  ggtitle("ML model", "FP counts, predicted as new images")
```

